Deployment Guide

Hosting Vite + PixiJS Projects on GitHub Pages (Sub-path)

This document contains the complete step-by-step workflow for successfully deploying a Vite project (like your PixiJS Slot Game) to a GitHub Pages sub-path, along with solutions for common missing-asset (404) errors.

Phase 1: Project Configuration

Step 1: Configure Vite for GitHub Pages

Vite needs to know it will be hosted at a sub-path instead of the root domain. Create or update `vite.config.ts` in the root of your project.

Using a conditional base path ensures it works both locally (during `npm run dev`) and in production on GitHub.

```
import { defineConfig } from 'vite';

export default defineConfig(({ command }) => {
  return {
    // Replace 'Pixi-Slot-Game' with your exact repo name
    base: command === 'build' ? '/Pixi-Slot-Game/' : '/',
  }
});
```

Step 2: Set up Deployment Scripts

Install the `gh-pages` package as a dev dependency to automate the pushing of your built files to the GitHub branch.

```
npm install gh-pages --save-dev
```

Then, update the `"scripts"` section in your `package.json`:

```
"scripts": {
  "dev": "vite",
  "build": "tsc && vite build",
  "preview": "vite preview",
  "predeploy": "npm run build",
  "deploy": "gh-pages -d dist"
}
```

Phase 2: Handling Static Assets & JSON

Step 3: Use the Public Folder for Dynamic Assets

Files that are fetched at runtime (like `config/gameConfig.json`) are not bundled by Vite automatically. You must use the `public` directory.

1. Create a folder named exactly `public` in your project root (next to `package.json`).
2. Move your `config` folder inside the `public` folder.

Your structure should look like this:

```
your-repo-name/
├── public/
│   └── config/
│       └── gameConfig.json
├── src/
├── package.json
└── vite.config.ts
```

Step 4: Fetch using BASE_URL

Update your fetch calls or asset loaders in your code to inject Vite's environment base URL. This dynamically adapts the path depending on if you are in local dev or production.

```
// In your game.ts or main.ts
const configResponse = await fetch(`${import.meta.env.BASE_URL}config/
gameConfig.json`);
```

Phase 3: Deployment

Step 5: Build and Deploy

Run the deployment script from your terminal:

```
npm run deploy
```

This command automatically runs the build step (creating the `dist` folder) and pushes the contents of `dist` to the `gh-pages` branch on GitHub.

Step 6: Enable GitHub Pages

1. Go to your repository on GitHub.
2. Navigate to **Settings** > **Pages**.
3. Under **Source**, select **Deploy from a branch**.
4. Set the branch to **gh-pages** and the folder to / **(root)**.
5. Save. Your site will be live at `https://<username>.github.io/<repo-name>/` within minutes.

Troubleshooting common Vite + GitHub Pages Issues

1. SyntaxError: Unexpected token '<', "
This happens when GitHub Pages cannot find your JSON file. Instead of the file, it returns a

custom 404 HTML error page. When your code tries to parse this HTML page as JSON, it crashes.
Fix: Ensure your JSON file is inside the `public` folder so Vite copies it to the `dist` folder during build.

2. The dist Folder is Missing locally
VS Code often hides the `dist` folder because it is usually listed in your `.gitignore` file. As long as your terminal says "build successful" and you see file sizes generated, the folder exists on your hard drive.

3. Works on Localhost, 404s on GitHub (Case Sensitivity)
Windows/Mac file systems are case-insensitive, but GitHub Pages (Linux) is strictly case-sensitive.
Fix: Check that your code matches the exact casing of your folders. If your folder is `Config` but you fetch `config/...`, it will break on GitHub.

4. Site not updating after deployment
GitHub Pages caches your site aggressively. If you just deployed, wait 2-3 minutes, then open your browser and press Ctrl + Shift + R (Cmd + Shift + R on Mac) to force a hard reload.